50 AÑOS

Universidad Manuela Beltrán

Acreditada en Alta Calidad Multicampus



LAPTICON

CÓMO FUNCIONA LA APP

TALLER DE COMPUTACIÓN MÓVIL

Universidad Manuela Beltrán
Acreditada en Alta Calidad Multicampus

TABLA DE CONTENIDO

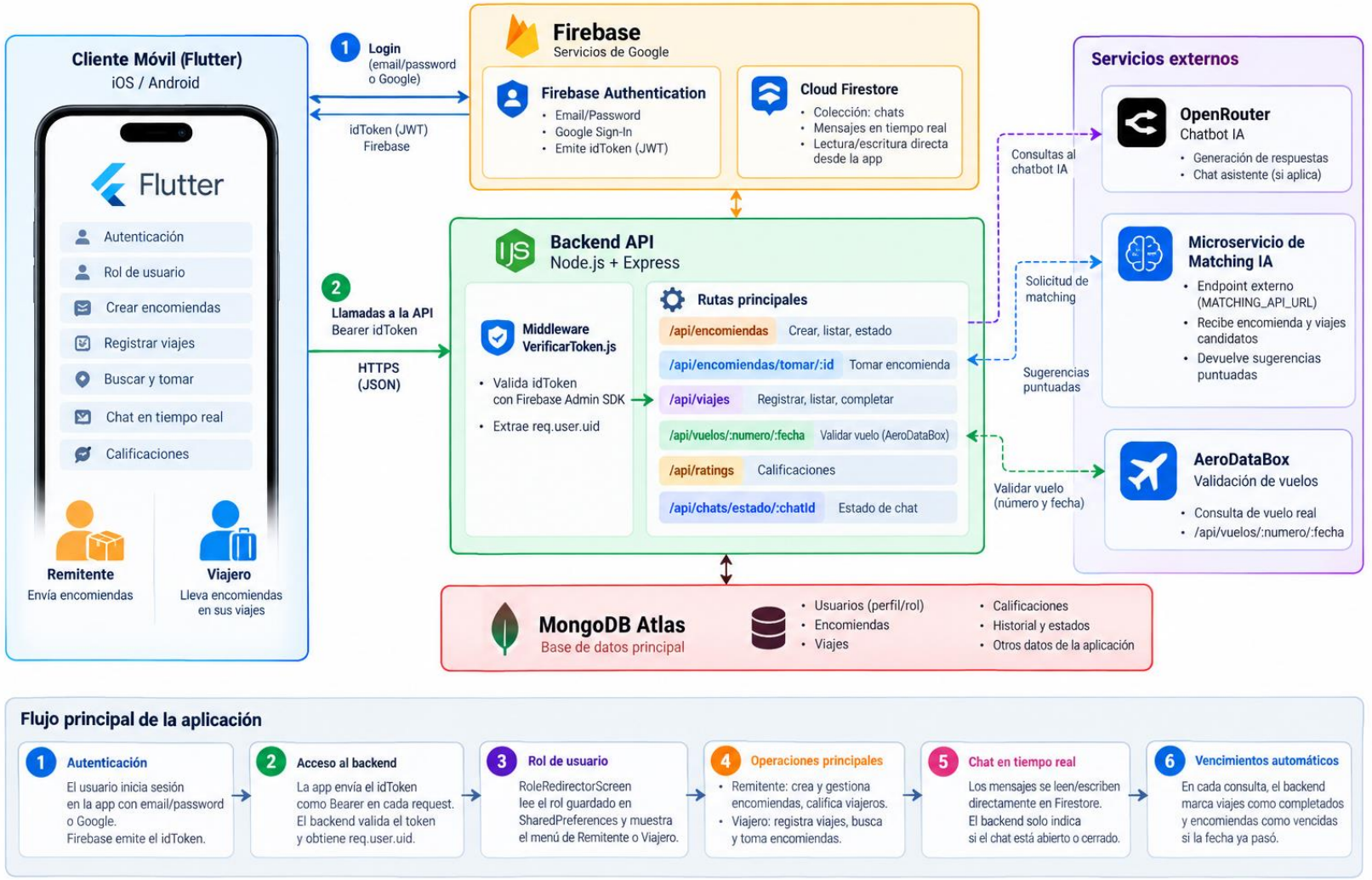
01 **ARQUITECTURA Y AUTENTICACIÓN**
Cómo se conectan Flutter, Firebase y el backend

02 **ROLES Y FLUJOS**
Remitente, viajero y el ciclo de una encomienda

03 **SERVICIOS**
Matching con IA, chat y vencimientos automáticos

04 **MANOS A LA OBRA**
Mapa de endpoints y retos para desarrollar

MANOS A LA OBRA



Flujo principal de la aplicación

1. **Autenticación** — El usuario inicia sesión en la app con email/password o Google. Firebase emite el idToken.
2. **Acceso al backend** — La app envía el idToken como Bearer en cada request. El backend valida el token y obtiene req.user.uid.
3. **Rol de usuario** — RoleRedirectorScreen lee el rol guardado en SharedPreferences y muestra el menú de Remitente o Viajero.
4. **Operaciones principales**
   - Remitente: crea y gestiona encomiendas, califica viajeros.
   - Viajero: registra viajes, busca y toma encomiendas.
5. **Chat en tiempo real** — Los mensajes se leen/escriben directamente en Firestore. El backend solo indica si el chat está abierto o cerrado.
6. **Vencimientos automáticos** — En cada consulta, el backend marca viajes como completados y encomiendas como vencidas si la fecha ya pasó.

Lapticon conecta a quien necesita enviar un paquete con un viajero que ya va a ese destino

OBJETIVO

Que cada equipo desarrolle una app móvil funcional que consuma el backend de Lapticon y cubra el ciclo completo de una encomienda.

CÓMO FUNCIONA

1. **Remitente:** publica una encomienda con origen, destino y fecha.
2. **Viajero:** registra su vuelo y toma una encomienda de su ruta.
3. **Entrega:** ambos se coordinan por chat y se califican al final.

TECNOLOGÍA LIBRE

El backend es el mismo; el cliente lo elige cada equipo.

- **Flutter**
- **PWA**
- **React Native**
- **Kotlin / Swift**
- **Otra pertinente**

Requisito: consumir la API REST con el idToken de Firebase.

¿QUÉ ES LAPTICON?

Lapticon une a quien envía un paquete con quien ya viaja a ese destino

REMITENTE
Quien envía un paquete

- Publica una **encomienda**: qué envía, ruta y fecha límite
- Sigue su estado: disponible, tomada o vencida
- **Califica** al viajero cuando recibe la entrega

la encomienda

viaja con el viajero

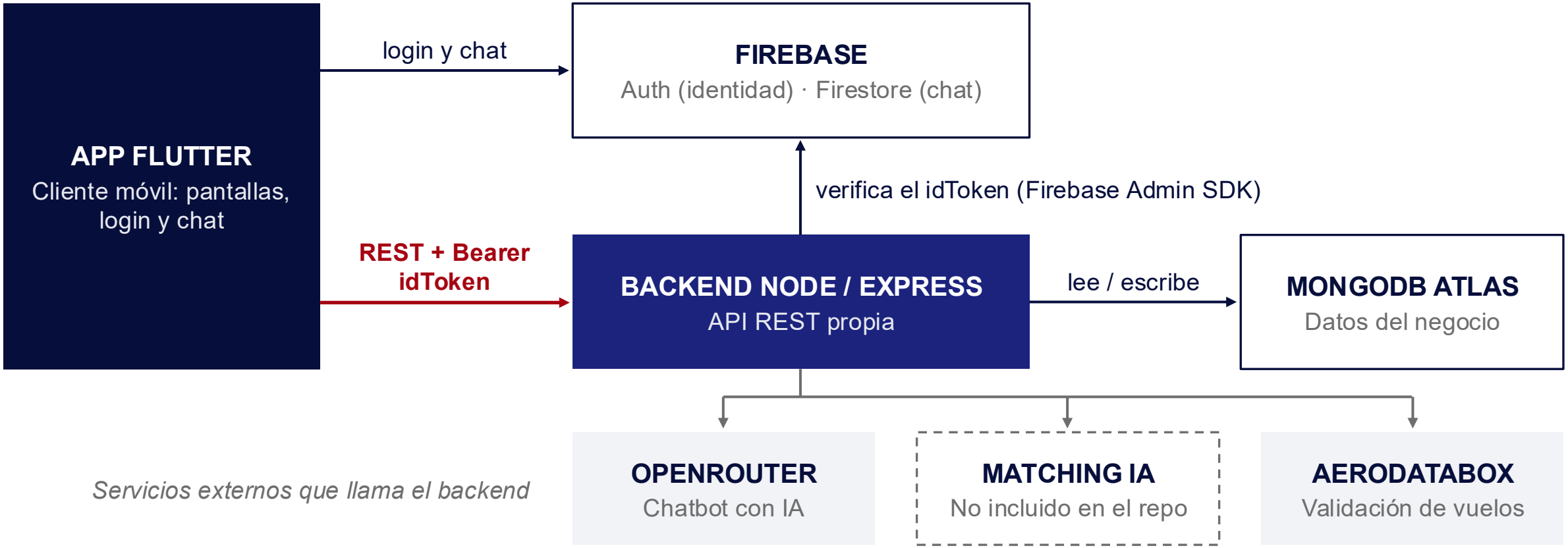
VIAJERO
Quien viaja y lleva el paquete

- Registra un **viaje** validado contra un vuelo real
- Busca encomiendas que coincidan con su ruta
- **Toma** la encomienda y la entrega en destino

La app los conecta: el matching con IA sugiere viajeros, un chat coordina la entrega y las calificaciones cierran el ciclo.

ARQUITECTURA Y AUTENTICACIÓN

La app habla con dos lados: Firebase para identidad y chat, y un backend propio para los datos



Firebase es el único proveedor de identidad: el backend verifica tokens y nunca emite sesiones

1 Inicio de sesión

AuthScreen usa FirebaseAuth: email y contraseña, o cuenta de Google.

```
AuthScreen
```

→

2 Firebase emite el idToken

Un token firmado por Firebase que identifica al usuario.

```
user.getIdToken()
```

→

3 Request al backend

ApiClient._getHeaders() agrega el token a cada llamada.

```
Authorization: Bearer <idToken>
```

→

4 Validación del token

VerificarToken.js lo valida con el Firebase Admin SDK.

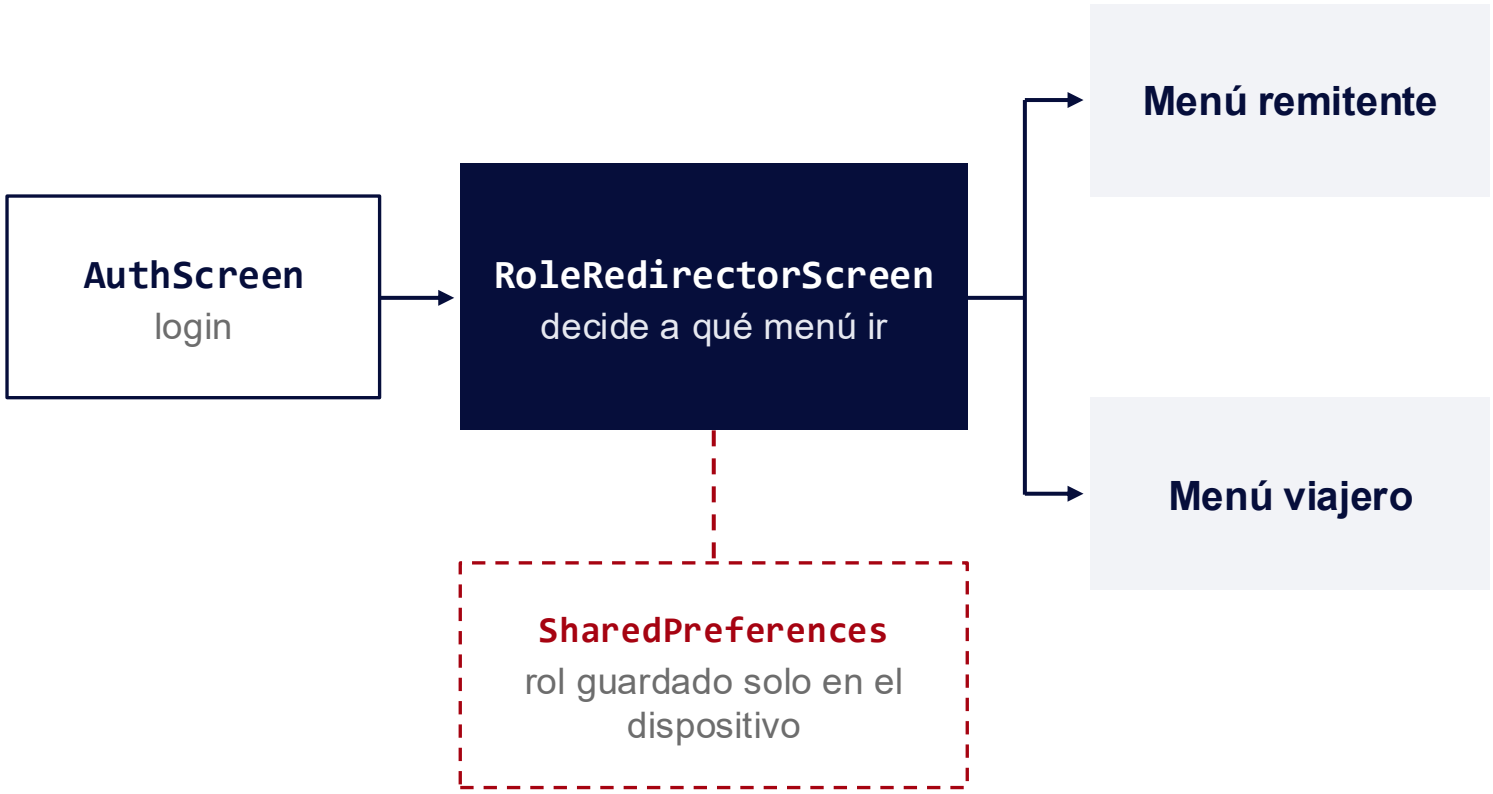
```
req.user.uid
```

El backend nunca crea sesiones propias: solo confía en tokens firmados por Firebase y lee el usuario desde req.user.uid.

Sin token o con un token inválido, **el middleware responde 401** antes de llegar a la ruta.

ROLES Y FLUJOS

Cada usuario entra como remitente o viajero, y hoy ese rol solo vive en el teléfono



RIESGO

El rol se pierde si el usuario cambia de dispositivo o borra los datos de la app.

PROPUESTA PARA EL TALLER

Guardar el rol en el perfil del usuario en el backend y usar SharedPreferences solo como caché.

El remitente publica una encomienda, sigue su estado y califica al viajero al recibirla

1. **Crea la encomienda**
   Define qué envía, la ruta y la fecha de vencimiento. Queda en estado disponible.

   `crear_encomienda_screen.dart`
   `POST /api/encomiendas`

2. **Sigue sus encomiendas**
   Las ve agrupadas en disponibles, tomadas y vencidas.

   Estados: disponible · tomada · vencida

3. **Un viajero la toma y la entrega**
   El viajero la vincula a su uid y luego la marca como entregada.

   Acción del viajero (ver siguiente diapositiva)

4. **Califica al viajero**
   Cuando la encomienda queda entregada, evalúa al viajero.

   `rating_form_screen.dart`
   `POST /api/ratings`

El viajero registra un vuelo real, toma encomiendas de su ruta y las entrega

1. **Registra su viaje**
   Ingresa el vuelo, la fecha y la ruta que va a recorrer.

   `viajero_form_screen.dart`
   `POST /api/viajes`

2. **El backend valida el vuelo**
   Consulta AeroDataBox para confirmar que el vuelo existe.

   `validación contra vuelo real`
   `/api/vuelos/:numero/:fecha`

3. **Busca encomiendas de su ruta**
   Ve las encomiendas disponibles que coinciden con su viaje.

   Sugerencias del matching IA (sección 03)

4. **Toma una encomienda**
   Pasa a estado tomada y queda vinculada a su uid.

   `cambia el estado a tomada`
   `PUT /api/encomiendas/tomar/:id`

5. **Entrega y la marca como entregada**
   El remitente confirma y califica al viajero.

   `actualiza el estado`
   `PUT /api/encomiendas/:id/estado`

CICLO DE VIDA DE UNA ENCOMIENDA

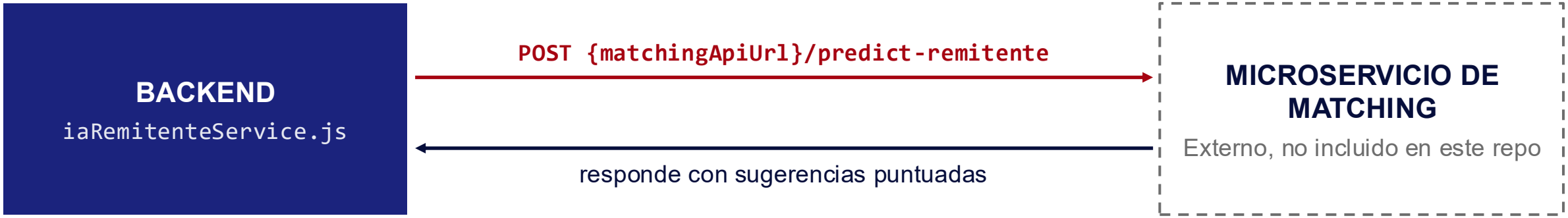
Una encomienda avanza hasta entregada o vence sola al pasar su fecha



`actualizarEncomiendasVencidas()` la marca como vencida la próxima vez que alguien consulta las encomiendas.

SERVICIOS: IA, CHAT Y VENCIMIENTOS

El backend no calcula el matching: lo delega a un microservicio externo



ENVÍA
La encomienda y la lista de viajes candidatos

RECIBE
Los viajes sugeridos, cada uno con su puntaje

En este entorno no funciona aún: `MATCHING_API_URL_DEV` y `MATCHING_API_URL_PROD` están vacíos en .env. Hay que configurar la URL del microservicio.

Los mensajes van directo de la app a Firestore; el backend solo sabe si el chat sigue abierto



ID DEL CHAT

Es uid1_uid2 con los dos uid ordenados. El mismo par de usuarios siempre abre el mismo documento, sin importar quién escriba primero.

CIERRE AUTOMÁTICO

`chatService.cerrarChatSiEntregaConfirmada`

cierra el chat cuando las dos partes califican después de la entrega.

Los vencimientos se aplican solo cuando alguien consulta la ruta: no hay un cron

Se crea

estado: disponible

Pasa la fechaVencimiento

En la base de datos no cambia nada

la base de datos muestra un estado viejo

Alguien consulta la ruta

El backend corre la función y la marca como vencida

ENCOMIENDAS

`actualizarEncomiendasVencidas()`

Marca como vencida cada encomienda con fecha pasada.

VIAJES

`actualizarViajesCompletados()`

Marca como completado cada viaje con fecha pasada.

Nueve rutas del backend sostienen los flujos principales de la app

Ruta	Para qué sirve	Quién la llama
`POST /api/encomiendas`	Crear una encomienda	Remitente
`PUT /api/encomiendas/tomar/:id`	Tomar una encomienda	Viajero
`PUT /api/encomiendas/:id/estado`	Cambiar el estado (p. ej. entregada)	Remitente o viajero asignado
`POST /api/viajes`	Registrar un viaje	Viajero
`/api/vuelos/:numero/:fecha`	Validar el vuelo con AeroDataBox	Viajero
`POST /api/ratings`	Calificar después de la entrega	Remitente y viajero
`/api/chats/estado/:chatId`	Saber si el chat sigue abierto	Remitente y viajero
`POST /api/chatbot`	Chatbot con IA (OpenRouter)	Usuario autenticado
`PUT /api/users/:uid`	Editar el perfil propio	Dueño del perfil

Las rutas protegidas exigen `Authorization: Bearer <idToken>` y responden 401 sin token; los mensajes del chat no pasan por el backend.

Retos para el taller



1 · Persistir el rol

Guardar el rol en el perfil del backend y leerlo al iniciar sesión; SharedPreferences queda como caché.



2 · Conectar el matching IA

Configurar MATCHING_API_URL y mostrar al remitente los viajes sugeridos con su puntaje.



3 · Vencimientos programados

Reemplazar la actualización lazy por una tarea programada que marque vencidas y completados.



4 · Chat completo

Escribir los mensajes en Firestore y bloquear el envío cuando el estado del chat sea cerrado.

Los estudiantes consultan aquí cada endpoint antes de programar una pantalla

DOCUMENTACIÓN DE LA API

Lapticon en Apiary

Rutas, peticiones y respuestas del backend en un solo lugar.

Abrir la documentación →

`claude.ai/artifact/3j2Hm5PgBJQGq25ye2Nuqz`

CÓMO USARLA EN EL TALLER

1. **Ubicar el endpoint**
   Buscar la ruta que llama la pantalla, p. ej. POST /api/encomiendas.

2. **Revisar el contrato**
   Comparar el cuerpo de la petición y la respuesta con el modelo en Flutter.

3. **Enviar el token**
   Agregar Authorization: Bearer <idToken>; sin él la ruta responde 401.

Seis prácticas incrementales: cada una agrega un módulo de la app y un concepto móvil

#	Práctica	Qué construyen	Qué aprenden
1	**Autenticación**	Login con Firebase (correo y Google) y primera llamada protegida	Identidad federada, idToken, Bearer
2	**Roles y navegación**	Selección de rol y menús por rol, con el rol guardado en servidor	Estado, rutas, persistencia local vs remota
3	**Encomiendas**	Formulario, listas por estado y POST/GET a /api/encomiendas	Consumo REST, validación, manejo de errores
4	**Viajes**	Registro de viaje validado contra un vuelo real	Servicios externos, asincronía, carga
5	**Chat**	Conversación remitente-viajero sobre Firestore	Tiempo real, streams, reglas de seguridad
6	**Entrega y calificación**	Marcar entrega, calificar y cerrar el chat	Flujo de extremo a extremo, pruebas

Al terminar, cada estudiante integra autenticación, API REST y tiempo real en una app móvil propia

RESULTADOS DE APRENDIZAJE

- **Diseña** una arquitectura cliente-servidor con servicios en la nube.
- **Implementa** autenticación con Firebase y consumo de una API protegida.
- **Integra** datos en tiempo real y servicios externos (vuelos, IA).
- **Argumenta** sus decisiones: tecnología, persistencia y manejo de fallos.

EVALUACIÓN SUGERIDA

40%	Prácticas 1 a 6, entregadas por etapa
30%	App final funcionando de extremo a extremo
20%	Demo y sustentación técnica
10%	Repositorio, código y documentación

Porcentajes de referencia; ajústelos al syllabus del curso.

Cinco entregas semanales llevan la app del login a la demo final

SEMANA 1

Práctica 1

Proyecto y repositorio. Login con Firebase y primera llamada protegida.

ENTREGA

Repo + login funcionando

SEMANA 2

Prácticas 2 y 3

Selección de rol y menús. Formulario para crear encomiendas.

ENTREGA

Roles + crear encomienda

SEMANA 3

Prácticas 3 y 4

Listas por estado. Registro de viaje validado y tomar encomienda.

ENTREGA

Flujo remitente-viajero

SEMANA 4

Prácticas 5 y 6

Chat en Firestore. Marcar entrega, calificar y cerrar el chat.

ENTREGA

Ciclo completo de la encomienda

SEMANA 5

Cierre

Pruebas, ajustes de interfaz, manejo de errores y documentación.

ENTREGA

App final + demo y sustentación

Cada entrega se revisa en clase sobre el repositorio y cuenta para el 40% de prácticas; la semana 5 cubre el 30% de app final y el 20% de sustentación.

Universidad Manuela Beltrán

Acreditada en Alta Calidad Multicampus

GRACIAS

¿PREGUNTAS?